

3rd REVIEW

Student Course Enrollment and Academic Management System

V. ABHIDEV	2520030325
TEJ PRATAP SINGH	2520030476
G. SHASHANK	2520030495

Under the Guidance
of

Dr. Y Subbarayudu

Assistant Professor
Department of Computer Science and Engineering



KONERU LAKSHMAIAH EDUCATION FOUNDATION

Project Overview

The portal manages course registration and academic records through one database. Students and administrators retrieve information through REST endpoints.

Students choose course sections within a selected semester cluster. The semester selection remains locked after enrollment or waitlisting.

Express services enforce eligibility, capacity and access permissions. Checks also apply to direct requests sent outside the interface.

Administrators manage marks, attendance and official CGPA with change history. Calculated GPA and an official override are distinct academic values.

The delivered release uses React, Node.js/Express and MySQL. Production serves the interface and API from one backend application.

Delivered Functional Scope

Area	Implemented capability
Student portal	Course catalog, enrollment planner, timetable and grades
Enrollment	Prerequisite checks, 24-credit limit, schedule conflicts and seat checks
Clusters	Four semester clusters with 2, 3, 2 and 3 section groups
Administration	Student directory with courses, grade publication and attendance
Academic reporting	Credit-weighted GPA, official CGPA and rankings
Record history	Enrollment notifications and academic audit records

Application Architecture

Layer	Implementation	Responsibility
Presentation	React pages and application context	User workflows and API requests
HTTP and access	Express routes and middleware	Authentication, roles and input validation
Business rules	Enrollment services	Eligibility, capacity and queue decisions
Data access	mysql2 connection pool	Parameterized SQL and transactions
Persistence	MySQL schema	Academic records, constraints and stored logic

Deployment model: one Express application serves the REST API and the production frontend.

Relational Data Model

Relationship	Database records	Rule
Student and registration	students, enrollments, courses	Enrollment links a student, course and section
Course and offering	courses, course_sections	Each offering belongs to one course
Cluster and group	academic_clusters, cluster_groups, section_clusters	Sections map to a cluster group
Student and semester choice	student_cluster_choices	One cluster selection per student and semester
Student and results	grades, attendance, cgpa_overrides	Academic records remain linked to the student

Foreign keys and uniqueness constraints protect relationships alongside application validation.

Enrollment Transaction and Consistency

Start a database transaction and lock section records in a stable order. The lock sequence reduces conflicting writes during registration.

Lock the student record and validate cluster, prerequisites and timetable compatibility. The operation also checks duplicate enrollment and the 24-credit limit.

Check section capacity and earlier eligible waitlist entries. The service decides whether to enroll, waitlist or reject the request.

Write enrollment, timetable and notification changes within the transaction. The semester cluster choice remains consistent with the new enrollment.

Commit successful changes or roll back the entire operation on failure. Related records remain unchanged when the operation fails.

Cluster Rules and Waitlist Handling

A student selects one cluster before requesting enrollment. Four clusters have 2, 3, 2 and 3 section groups.

The backend accepts only sections within that cluster and the current semester. Different subjects may use different groups inside the chosen cluster.

Enrollment or waitlisting locks the semester choice, including after a later drop. Students cannot reset the choice by dropping all their courses.

A full section can accept a waitlist request from an eligible student. A queue entry does not guarantee later enrollment.

When a seat opens, the service rechecks eligibility before promoting a waiting student. The service can skip an ineligible entry and consider a later eligible student.

Security and Request Validation

Salted scrypt hashes protect passwords stored in the database. The application does not store the entered password as plain text.

Signed JWT sessions expire after eight hours. Protected API requests include the token in the Authorization header.

Ownership checks restrict student access to personal records. The backend verifies identity before reading protected student data.

Administrator permissions protect marks, attendance and official CGPA changes. Direct API requests receive the same role checks as interface actions.

Parameterized queries, request limits and authentication rate limiting reduce common risks. The JSON body limit is 64 KB and the backend sets security headers.

Academic Administration and Audit History

The student directory brings enrolled courses and academic details into one view. The administrator can inspect the student's selected cluster as well.

Published grades produce a GPA weighted by course credits. Semester GPA filters results to the student's current semester.

An official CGPA override accepts a value from 0 to 10 and requires a reason. The effective CGPA uses this official value when an override exists.

Removing the override restores the grade-derived calculation. Published grades remain available for the underlying calculation.

The audit history records academic changes for administrative review. Enrollment events also support investigation of registration changes.

REST API Contract

Method	Endpoint	Purpose
POST	/api/auth/login	Issue an authenticated session
GET	/api/courses	Read course offerings
POST	/api/enrollments	Enroll or join a section waitlist
PUT	/api/clusters/students/:id	Choose the student's cluster
PUT	/api/clusters/students/:id/cgpa	Set or clear official CGPA
GET	/api/admin/overview	Read administrator statistics

OpenAPI contract: </api/openapi.json>. Protected requests use a Bearer token.

API and Database Demonstration

Demonstration	API evidence	Database or permission evidence
Student login	JWT returned in JSON	Student identity and role
Course retrieval	Course catalog response	Compare with <code>SELECT * FROM courses</code>
Enrollment	Accepted or rejected request	Inspect enrollments and timetable records
Cluster restriction	Cross-cluster request rejected	No invalid enrollment or seat loss
Official CGPA update	Admin update response	<code>cgpa_overrides</code> and <code>academic_audit</code>
Protected admin route	Student receives 403 Forbidden	Admin token grants authorized access

Verification Results

Check	Recorded result	Coverage
Core unit tests	7 passed	Password handling, JWT and enrollment rules
Database/API checks	40 passed	Enrollment, clusters, waitlists, CGPA and audit
React component checks	8 passed	Student/admin pages and enrollment states
Production frontend build	Passed	React application builds for production
Repeated database setup	Preserved existing records	Setup does not reset academic data

Recorded environment: Node.js 24 and MariaDB 10.11.14. Intended deployment: MySQL 8.0.16+.

Engineering Tradeoffs and Production Readiness

Current decision	Engineering benefit	Production follow-up
One backend application	Simple local execution and debugging	Measure workload before separating services
Section locks during enrollment	Consistent seats and queue promotion	Benchmark and narrow lock scope if needed
JWT sessions and application roles	Controlled access to student/admin operations	Institutional SSO and session revocation
Local database and startup scripts	Repeatable development setup	Backups, restore tests and managed hosting
Automated core and API checks	Regression coverage for key rules	Windows, MySQL and browser end-to-end testing

Alignment with the Backend Handout

Outcome	Project connection	Coverage
CO1	Relational modeling, SQL and transactions	Implemented with MySQL
CO3	REST, OpenAPI, JWT, hashing and roles	API/security concepts implemented
CO4	Node.js, Express, validation and middleware	Node/Express framework portion implemented
CO2, CO5, CO6	Additional databases, distributed services and operations	Outside the current delivered scope

The handout lists PostgreSQL. The project uses MySQL as its relational database.

Conclusion and Development Roadmap

The release provides a persistent enrollment and academic administration workflow. Students and administrators operate on the same relational records.

Transactions protect registration data while server rules enforce eligibility and clusters. Cluster locks persist across later course drops in the semester.

Administrative permissions and audit history support accountable academic changes. Official CGPA changes require a validated value and a reason.

The next release should prioritize deployment, backups and complete end-to-end testing. HTTPS, recovery procedures and operational monitoring need deployment work.

Workload measurements should guide later capacity and architecture improvements. The current section locking strategy prioritizes consistency over throughput.

Live Project Demonstration

Step	Action	Evidence
1	Start backend and inspect health	API running and database connection available
2	Choose a cluster and enroll	Accepted section appears in the timetable
3	Request a different-cluster section	Backend rejects the incompatible request
4	Admin updates official CGPA	Updated academic summary and change reason
5	Release a full section's seat	Eligible waiting student receives enrollment
6	Run matching Workbench queries	Persisted records confirm application changes

Postman supplements the demonstration with HTTP requests and JSON responses.

Results and Project Outcomes

Delivered result	Evidence	Practical outcome
Cluster consistency	Cross-cluster requests rejected	Subjects remain within one semester choice
Validated registration	Eligibility and capacity checks	Invalid enrollment leaves no partial record
Academic visibility	Directory and summary views	Administrators inspect students and subjects
Official CGPA control	Validated override with a reason	Academic updates retain a change history
Waitlist support	Eligibility recheck on seat release	Available seats can reach waiting students
Persistent operation	SQL records and audit entries	Changes survive interface refreshes

Documentation and Repository Deliverables

Artifact	Purpose in final evaluation
Project source	React interface, Express routes and enrollment services
README.md	Setup, startup, API documentation and demo sequence
Database scripts	Schema, demonstration data and repeatable database setup
Postman collection and OpenAPI	API requests, authorization and documented contracts
Verification record	Recorded test results with environment limitations
Review decks and poster	Problem, implementation, architecture and outcomes

Repository: github.com/Tejpratap-069/dbse-dbd-projects Folder: Project

THANK YOU

Questions and Live Demonstration